SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO5: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)

Assessment Tool Weightage: 35%

Dr G A. SENTHIL

QUESTIONS: 10 ***Total Marks: 50***

Code of Conduct:

I **__T Bhanu Teja/(192472259)___** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Bhanu Teja

Annexure A

Mock Interview

1. Personalized Notification System Using Reinforcement Learning

- **AIM**

To develop an RL-based personalized notification system that learns the best time and type of notification for improving user engagement.

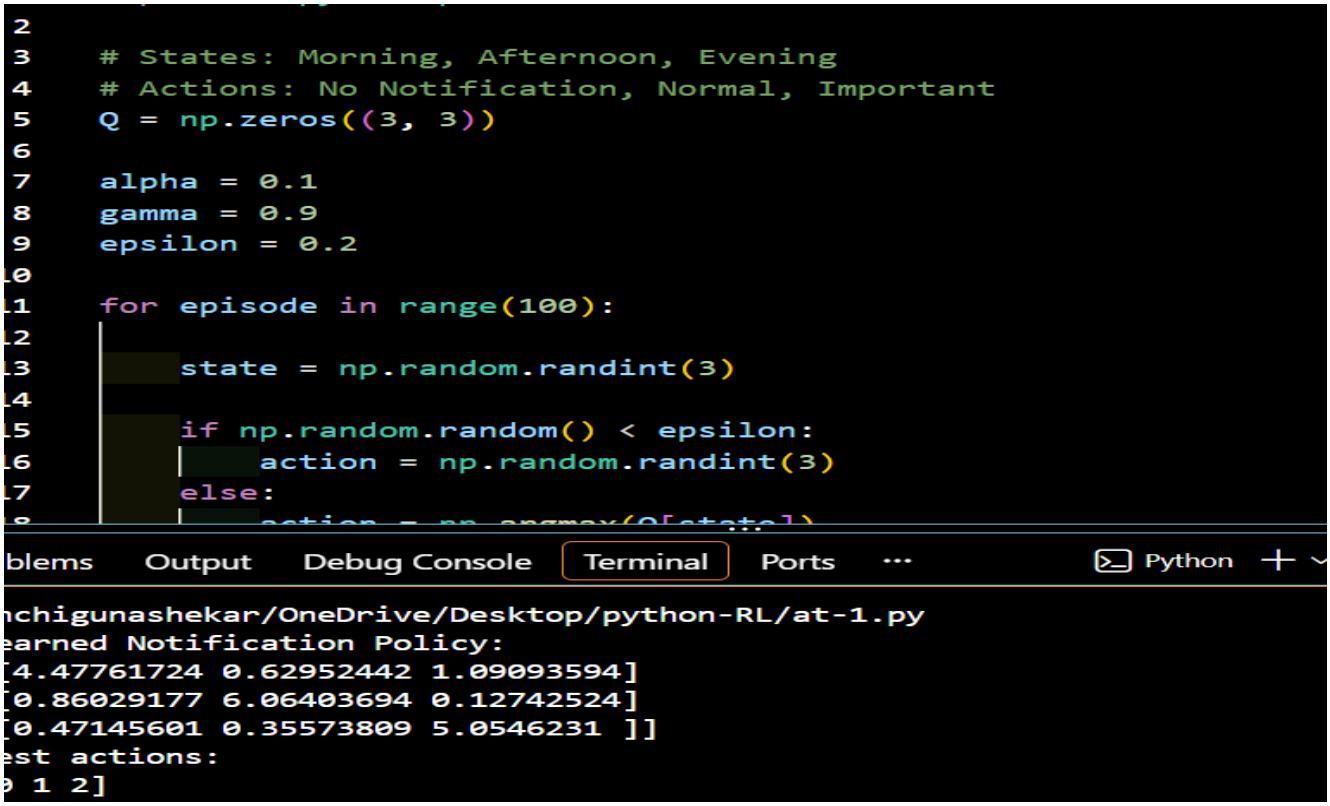
- ABSTRACT

The mobile application treats the user as an environment and the RL agent learns from user interactions. The state contains information such as time, user activity, previous notification response, and app usage. The actions represent different notification choices, while rewards are given based on whether the user opens, clicks, ignores, or disables the notification.

- **ALGORITHM**

1. Initialize the RL agent.
2. Collect user activity and notification interaction data.
3. Define the current state using time, activity, and previous response.
4. Select a notification action using an ε-greedy policy.
5. Send the selected notification.
6. Observe the user's response.
7. Assign a reward based on the response.
8. Update the Q-value.
9. Repeat for multiple user interactions.
10. Select the notification strategy with the highest expected reward.

- **OUTPUT:-**



- **CONCLUSION:-**

The RL framework can learn personalized notification strategies by continuously observing user behavior and adjusting notification timing and type. A suitable reward function encourages useful notifications while discouraging unwanted notifications.

2. Warehouse Robot – Exploration vs Exploitation

- **AIM**

To analyze the effect of exploration and exploitation on warehouse robot navigation and improve the robot's ability to find shorter paths.

- **ABSTRACT**

A warehouse robot must balance exploration, where it tries new paths, and exploitation, where it follows paths already known to be successful. If exploration is too low, the robot may repeatedly select a longer path and fail to discover a shorter route. An ε-greedy strategy can be used to control this trade-off.

- **ALGORITHM**

1. Create a warehouse grid containing obstacles.
2. Define the robot's start and destination.
3. Initialize the Q-table.
4. Select actions using ε-greedy exploration.
5. Give positive reward for reaching the destination.

6. Give negative reward for collisions and unnecessary movement.
7. Update the Q-values.
8. Gradually reduce ε during training.
9. Continue training for multiple episodes.
10. Use the learned best actions during final navigation.

- **OUTPUT:-**

```
import numpy as np

Q = np.zeros((5, 4))

alpha = 0.1
gamma = 0.9
epsilon = 1.0

for episode in range(100):

    state = 0

    for step in range(10):

        # Exploration vs Exploitation
        if np.random.random() < epsilon:
            action = np.random.randint(4)
```

```
Problems  Output  Debug Console  Terminal  Ports  ...  Python

anchigunashekar/OneDrive/Desktop/python-RL/AT-2.PY
Learned Q-Table:
[[-0.59918493  0.21585064  4.41333784 -0.47563656]
 [ 1.69646471  0.8820766   0.98147544  6.14024043]
 [ 1.80363589  2.01216216  7.98556596  2.74249936]
 [ 4.0951      4.0951      4.68559     9.99856659]
 [ 0.          0.          0.          0.        ]]
```

- **CONCLUSION:-**

Insufficient exploration can cause the warehouse robot to select longer paths because it does not discover better alternatives. Using ε-greedy exploration, reward shaping, and gradual reduction of ε allows the robot to explore initially and exploit the shortest learned path later.

3. Hospital Patient Appointment Scheduling Using RL

- **AIM**

To demonstrate the suitability of Reinforcement Learning for optimizing hospital appointment scheduling and reducing patient waiting time.

- **ABSTRACT**

In a hospital scheduling system, RL can learn how to assign patients to doctors, rooms, and time slots. The state can contain patient waiting time, doctor availability, room availability, and appointment queue length. The agent receives rewards for reducing waiting time and efficiently utilizing hospital resources.

- **ALGORITHM**

1. Collect appointment and resource availability information.
2. Define the hospital scheduling state.
3. Define possible scheduling actions.
4. Assign a reward for each scheduling decision.
5. Give positive rewards for reduced waiting time.

6. Give penalties for long waiting times and resource conflicts.
7. Update the RL policy using observed results.
8. Consider delayed outcomes such as treatment completion.
9. Evaluate scheduling efficiency.
10. Apply ethical constraints before deploying the system.

- **OUTPUT:-**

```
import numpy as np

# States: Low, Medium, High waiting queue
# Actions: Doctor 1, Doctor 2, Doctor 3
Q = np.zeros((3, 3))

alpha = 0.1
gamma = 0.9
epsilon = 0.1

for episode in range(100):

    state = np.random.randint(3)

    if np.random.random() < epsilon:
        action = np.random.randint(3)
    else:
```

```
Problems  Output  Debug Console  Terminal  Ports  ...  Python

C:\Users\Madamanchigunashekar\OneDrive\Desktop\python-RL>C:\Users\Madamanchi
gunashekar\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Madam
anchigunashekar/OneDrive/Desktop/python-RL/AT3.PY
Appointment Scheduling Q-Table:
[[63.29419395  0.          0.        ]
 [74.88775745 13.18547139  3.08762645]
 [71.14232489 11.1921697   0.        ]]
```

- **CONCLUSION:-**

RL is suitable for appointment scheduling because it can continuously adapt to changing patient demand and resource availability. However, delayed rewards, changing hospital conditions, fairness, patient privacy, safety, and ethical decision-making must be considered before real-world deployment.

4. Smart Home Energy Optimization Using RL

- **AIM**

To design an RL-based smart home system that reduces electricity consumption by automatically controlling lights, fans, and air conditioners.

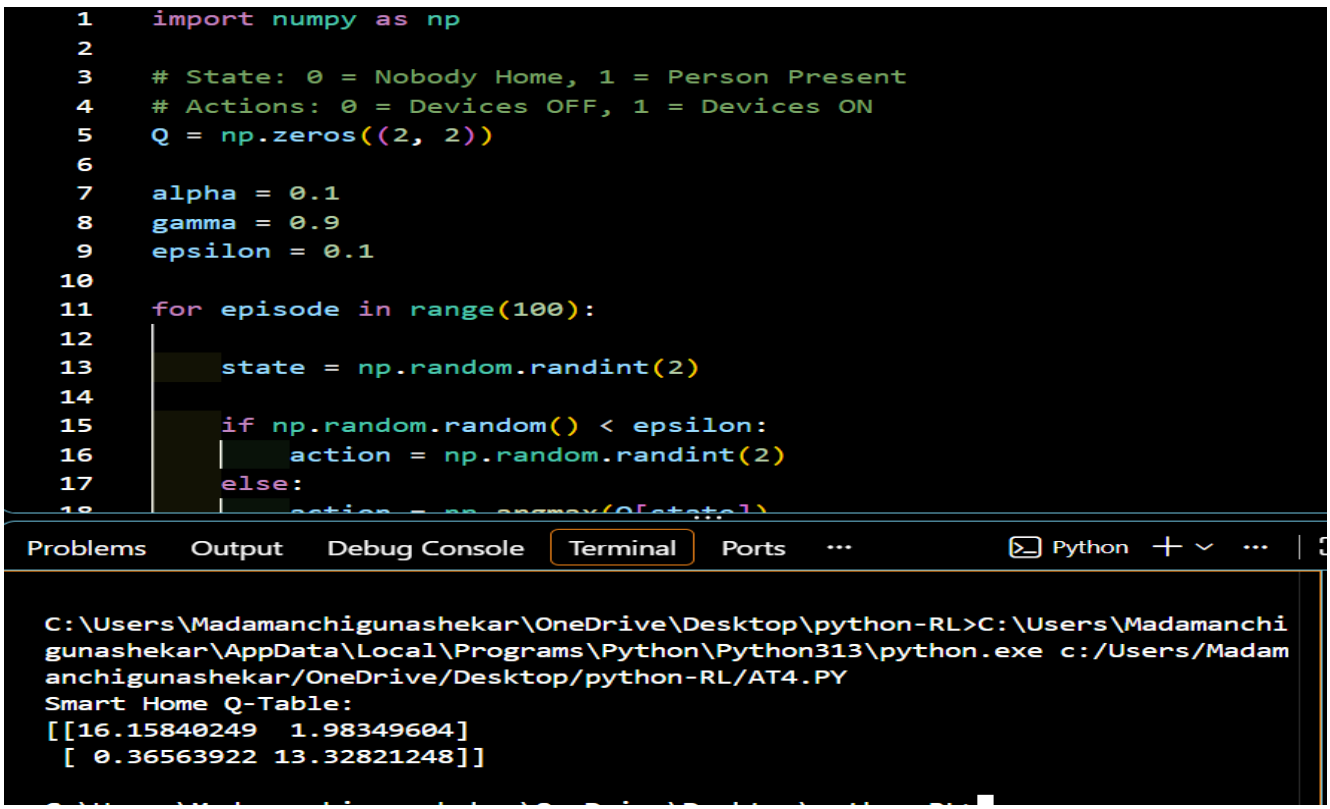
- **ABSTRACT**

The smart home system uses RL to learn energy-efficient device control. The state contains user presence, temperature, weather conditions, time, and current energy consumption. The actions control household devices. The reward encourages lower electricity consumption while maintaining user comfort.

- **ALGORITHM**

1. Initialize the smart home environment.
2. Define the state space.
3. Define the device control actions.
4. Monitor temperature, weather, presence, and energy usage.
5. Select an action using an RL algorithm.
6. Apply the selected device settings.
7. Calculate energy consumption and comfort level.
8. Give positive rewards for energy savings.
9. Give penalties for excessive energy use or poor comfort.
10. Update the agent and repeat.

- **OUTPUT:-**



- **CONCLUSION:-**

The RL model can learn energy-efficient control policies for smart home devices. By combining energy consumption and user comfort in the reward function, the system can reduce unnecessary electricity usage while maintaining a comfortable environment.

5. RL-Based Online Game Matchmaking

- **AIM**

To develop an RL-based matchmaking system that matches online players with similar skill levels while maintaining player engagement and fairness.

- **ABSTRACT**

The RL agent observes player skill rating, recent performance, waiting time, and previous match results. It selects an opponent or group of opponents. The reward should encourage fair matches, suitable skill differences, reasonable waiting time, and long-term player engagement. Poor reward design can cause biased matchmaking or prioritize short waiting times over fair competition.

- **ALGORITHM**

1. Collect player skill and performance information.
2. Define the matchmaking state.
3. Generate possible opponent groups.
4. Select a matching action using the RL policy.
5. Observe the match result.
6. Calculate skill difference and player satisfaction.
7. Give rewards for fair and engaging matches.
8. Penalize large skill differences and excessive waiting.
9. Update the RL policy.
10. Continuously evaluate fairness and player engagement.

- **OUTPUT:-**

```python
import numpy as np

# States represent player skill levels
Q = np.zeros((3, 3))

alpha = 0.1
gamma = 0.9
epsilon = 0.1

for episode in range(100):

    player_skill = np.random.randint(3)

    if np.random.random() < epsilon:
        opponent = np.random.randint(3)
    else:
        opponent = np.argmax(Q[player_skill])
```

Problems Output Debug Console Terminal Ports ··· Python

```
anchigunashekar/OneDrive/Desktop/python-RL/AT5.PY
Matchmaking Q-Table:
[[ 9.60395006  0.          0.        ]
 [ 6.48234351  0.82414038  1.32600763]
 [-0.04235583  7.05728405  1.12970694]]
Best opponent for each skill level:
[0 0 1]
```

- **CONCLUSION**

The MARL system allows multiple robots to learn and perform path planning cooperatively. By considering other robots' positions and using rewards and penalties, the system can reduce collisions and improve overall navigation efficiency.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge

		understanding			
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrate d
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicate s well with minor hesitation	Communication lacks clarity or confidence	Unable to communicat e effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionali sm
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately